

Coding & Solution

Date	01 JULY 2026
Team ID	XXXXX
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Harshitha-17-stack/OptiCrop-Smart-Agricultural-Production-Optimization-Engine
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FLASK
Key Features Implemented	1. Automated Asset Build Pipeline (build_assets.py) A single end-to-end pipeline that performs: • Data loading • Feature engineering • Preprocessing pipeline fitting • Model training and serialization 2. Crop Target Engineering (src/data_pipeline.py) • Converts raw agricultural data into structured ML format • Creates crop recommendation labels based on environmental conditions (N, P, K, temperature, humidity, rainfall) 3. Serialized Preprocessing Pipeline (src/data_preprocessing.py) • Implements Scikit-learn ColumnTransformer • Handles: - Scaling numerical features - Encoding categorical values - Missing value handling • Saved as reusable .joblib pipeline for inference

	4. Interactive Web Dashboard (app.py, templates/index.html) • Flask-based UI for user interaction • Features: Crop input form (soil + climate parameters) Real-time prediction output and result display (recommended crop)
Pending / Incomplete Features	1. Centralized Configuration System Hardcoded parameters (file paths, model configs, etc.) Improvement: Move to config.yaml or .env file 2. Unit & Integration Testing No automated test suite implemented yet Recommendation: Add pytest for pipeline and API testing 3. Logging & Monitoring System Current logs printed to console only Improvement: Implement structured logging.
Setup / Run Instructions	1. Activate Environment: Set up virtual environment (python -m venv venv) and activate (.venv\Scripts\Activate.ps1). 2. Install Packages: Install dependencies: pip install -r requirements.txt. 3. Build Assets: Run the full assets pipeline script: python build_assets.py. 4. Start Web Server: Launch Flask app: python app.py and open http://127.0.0.1:5000 in your browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	YES
2	Meaningful variable and function names are used	YES
3	Code includes comments / documentation where necessary	YES
4	Error handling is implemented for critical operations	YES
5	The application runs without critical errors	YES
6	Code is committed to a version control repository	YES

Additional Notes / Comments:

1. Machine Learning

- Handles class imbalance using `class_weight="balanced"`
- Prevents data leakage using fitted preprocessing pipeline
- Uses train-test split validation strategy

2. Error Handling

Try-except blocks used for:

- File loading
- Model inference
- Input parsing in Flask routes

3. Version Control

- Git initialized at project root
- `.gitignore` used to exclude:
- Large datasets
- `.pkl` / `.joblib` model files
- Cache files